

CS 577 — Deep Learning — Homework 5

Read these instructions carefully:

- In the L^AT_EX source code, type your answer in between “%% BEGIN ANSWER” and “%% END ANSWER”. For advanced L^AT_EX users, you can use your custom macros if you wish by placing them between “%% BEGIN MACROS” and “%% END MACROS” in the header. Do not modify anything else.
- Turn in both your .tex file and the generated .pdf file.

1 The transformer module

In homework 3 and 4, we investigated computation for a simple attention model, where each data point $\mathbf{X}^{(i)}$ is itself a *collection* of C vectors (for concreteness, take $C = 32$). Namely, we write $\mathbf{X}^{(i)}$ as a matrix whose c -th column is $\mathbf{x}^{(i,c)}$, i.e.,

$$\mathbf{X}^{(i)} = [\mathbf{x}^{(i,1)} \quad \dots \quad \mathbf{x}^{(i,C)}] \in \mathbb{R}^{d \times C} \quad \text{is a } d \times C \text{ matrix}$$

You can think of the $\mathbf{x}^{(i,j)}$ ’s for each $j = 1, \dots, C$ as the “word embedding” for some sentence of length C . We say that $\mathbf{X}^{(i)}$ is a “sequence”.

In the code, we will name

- C as `n_context`
- d as `n_features`
- n as `n_samples`, which denotes the mini-batch size $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}$.

The transformer module that we consider in this homework is composed of a *self-attention* submodule followed by a *multilayer perceptron* submodule with 1-hidden layer. For simplicity, we focus on these two submodules, leaving aside for now dropout, layer norm, etc. Both of these submodules will have parameters. We will refer to these parameters as $\theta^{(\text{att})}$ and $\theta^{(\text{mlp})}$ respectively.

The self-attention submodule has parameters

$$\theta^{(\text{att})} = [\mathbf{W}^{(Q)}, \mathbf{W}^{(K)}, \mathbf{W}^{(V)}] \quad (“Q, K, V” \text{ stands for “query”, “key”, “value”, respectively})$$

where

$$\mathbf{W}^{(Q)} \text{ and } \mathbf{W}^{(K)} \text{ and } \mathbf{W}^{(V)} \in \mathbb{R}^{d \times d}.$$

Here are the main changes compared to the previous homeworks

- We assume $q = d$ for simplicity. In this homework, there is no separate `n_reduced`.
- In previous homeworks, we had $\mathbf{W}^{(1)}$ instead of $\mathbf{W}^{(Q)}$, and had $\mathbf{W}^{(2)}$ instead of $\mathbf{W}^{(K)}$.
- In previous homeworks, we had $\mathbf{w}^{(3)}$ which was a vector. In this homework, we have that $\mathbf{W}^{(V)}$ is a $d \times d$ matrix.

In this homework, our self-attention submodule is “seq-2-seq”, i.e., it maps the sequence $\mathbf{X}^{(i)}$ to another sequence $\text{attention}(\mathbf{X}^{(i)}; \theta)$:

$$\text{attention}(\mathbf{X}^{(i)}; \theta) := \mathbf{W}^{(V)\top} \mathbf{X}^{(i)} \text{softmax} \left(\mathbf{X}^{(i)\top} \mathbf{W}^{(K)\top} \mathbf{W}^{(Q)} \mathbf{X}^{(i)} \right) \in \mathbb{R}^{d \times C}.$$

Note that

- $\text{softmax} \left(\mathbf{X}^{(i)\top} \mathbf{W}^{(K)\top} \mathbf{W}^{(Q)} \mathbf{X}^{(i)} \right)$ is a C -by- C matrix.
- $\text{Attention}(\mathbf{X}^{(i)}; \theta)$ has the same shape as $\mathbf{X}^{(i)}$

The **MLP submodule** has a single hidden layer and the following parameters:

$$\theta^{(\text{mlp})} = [\mathbf{W}^{(H)}, \mathbf{b}^{(H)}, \mathbf{W}^{(O)}, \mathbf{b}^{(O)}] \quad (“H, O” \text{ stands for “hidden” and “output”, respectively})$$

where

$$\mathbf{W}^{(H)} \in \mathbb{R}^{d \times h}, \quad \mathbf{b}^{(H)} \in \mathbb{R}^h, \quad \mathbf{W}^{(O)} \in \mathbb{R}^{h \times d}, \quad \text{and} \quad \mathbf{b}^{(O)} \in \mathbb{R}^d.$$

The hidden-layer activation is computed as $\mathbf{H} = \text{relu}(\mathbf{W}^{(H)\top} \mathbf{X}^{(i)} + \mathbf{b}^{(H)}) \in \mathbb{R}^{h \times C}$ and the output

$$\text{MLP}(\mathbf{X}^{(i)}; \theta^{(\text{mlp})}) = \mathbf{W}^{(O)\top} \mathbf{H} + \mathbf{b}^{(O)} \in \mathbb{R}^{d \times C}$$

Putting both submodules together, we have the transformer block:

$$\text{TransformerBlock}(\mathbf{X}^{(i)}; \theta^{(\text{att})}, \theta^{(\text{mlp})}) := \text{MLP}(\text{Attention}(\mathbf{X}^{(i)}; \theta^{(\text{att})}); \theta^{(\text{mlp})})$$

The above can be implemented (vectorizing over a batch $\mathbf{X}^{(1)}, \dots, \mathbf{X}^{(n)}$) as follows:

```

1 # IMPORTANT: the batch data tensor has shape (n_samples, n_context, n_features)
2 #
3 class SingleHeadAttention:
4     def __init__(self, n_features):
5         self.Wq = ag.Tensor(np.random.randn(n_features, n_features), label="Wq")
6         self.Wk = ag.Tensor(np.random.randn(n_features, n_features), label="Wk")
7         self.Wv = ag.Tensor(np.random.randn(n_features, n_features), label="Wv")
8     def __call__(self, Xin):
9         Queries = Xin @ self.Wq
10        Keys = Xin @ self.Wk
11        KQ = (Keys @ ag.moveaxis(Queries, 1, 2))
12        expKQ = ag.exp(KQ)
13        softmaxKQ = expKQ / ag.sum(expKQ, axis=1, keepdims=True)
14        Xout = ag.moveaxis(ag.moveaxis(Xin, 1, 2) @ softmaxKQ, 1, 2) @ self.Wv
15        return Xout
16
17 class MLP:
18     def __init__(self, n_features, n_hidden):
19         self.Wh = ag.Tensor(np.random.randn(n_features, n_hidden), label="Wh")
20         self.bh = ag.Tensor(np.random.randn(n_hidden), label="bh")
21         self.wo = ag.Tensor(np.random.randn(n_hidden, n_features), label="Wo")
22         self.bo = ag.Tensor(np.random.randn(n_features), label="bo")
23
24     def __call__(self, Xin):
25         hidden = ag.relu((Xin @ self.Wh) + self.bh)
26         return hidden @ self.wo + self.bo
27
28 class TransformerBlock:
29     def __init__(self, n_features, n_hidden):
30         self.att = SingleHeadAttention(n_features)
31         self.mlp = MLP(n_features, n_hidden)
32     def __call__(self, Xin):
33         return self.mlp(self.att(Xin))

```

[5] point(s) — part a:

This is a programming exercise. See [hw5.ipynb](#)

[5] point(s) — part b:

Explain your answer from part a.

Answer:

[10] point(s) — part c:

Given that a single `matmul` between matrices of shape (a, b) and (b, c) requires $O(abc)$ floating point operations (FLOPs), compute the number of FLOPs required to compute one forward pass of the `TransformerBlock` on an input tensor of shape (n, C, d) . Give your answer in terms of big-O notation involving the quantities n, C, d and h .

Answer: